



Article

Refining Zero-Shot Text-to-SQL Benchmarks via Prompt Strategies with Large Language Models

Ruikang Zhou ¹ and Fan Zhang ^{2,3,*}

¹ School of Computation, Information and Technology, Technical University of Munich, 80333 Munich, Germany; ge74kuh@mytum.de

² State Key Laboratory of Ocean Sensing & Ocean College, Zhejiang University, Zhoushan 316021, China

³ Kavli Institute for Astrophysics and Space Research Center, Massachusetts Institute of Technology, Cambridge, MA 02139, USA

* Correspondence: f.zhang@zju.edu.cn

Abstract: Text-to-SQL leverages large language models (LLMs) for natural language database queries, yet existing benchmarks like BIRD (12,751 question–SQL pairs, 95 databases) suffer from inconsistencies—e.g., 30% of queries misalign with SQL outputs—and ambiguities that impair LLM evaluation. This study refines such datasets by distilling logically sound question–SQL pairs and enhancing table schemas, yielding a benchmark of 146 high-complexity tasks across 11 domains. We assess GPT-4o, GPT-4o-Mini, Qwen-2.5-Instruct, llama 370b, DPSK-v3 and O1-Preview in zero-shot scenarios, achieving average accuracies of 51.23%, 41.65%, 44.25%, 47.80%, and 49.10% and a peak of 78.08% (O1-Preview), respectively. Prompt-based strategies improve performance by up to 4.78%, addressing issues like poor domain adaptability and inconsistent training data interpretation. Error-annotated datasets further reveal LLM limitations. This refined benchmark ensures robust evaluation of logical reasoning, supporting reliable NLP-driven database systems.

Keywords: Text-to-SQL; Natural Language Processing (NLP); large language models (LLMs); zero-shot prompting; dataset benchmarking



Academic Editor: Andrea Prati

Received: 2 April 2025

Revised: 28 April 2025

Accepted: 5 May 2025

Published: 9 May 2025

Citation: Zhou, R.; Zhang, F. Refining Zero-Shot Text-to-SQL Benchmarks via Prompt Strategies with Large Language Models. *Appl. Sci.* **2025**, *15*, 5306. <https://doi.org/10.3390/app15105306>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The field of Text-to-SQL, which enables natural language querying of databases, has evolved significantly, propelled by neural architectures and large language models (LLMs). Early systems relied on inflexible rule-based or template-based methods, succeeded by neural approaches like Seq2SQL [1] and SQLNet [2], which introduced reinforcement learning and sequence-to-sequence models to improve SQL generation [3]. Modern solutions, such as DAIL-SQL [4] and SQL-PaLM [5], leverage LLMs with pretraining on vast text corpora and dynamic context learning, achieving high accuracy on benchmarks. While SQL's structured format enhances LLM efficiency over unstructured data [6], challenges persist in handling complex queries, ensuring cross-domain generalization—where performance consistency falters across diverse domains—and maintaining accuracy amid ambiguous inputs [7].

Zero-shot prompting, where LLMs perform tasks without task-specific training or examples, has gained traction in Text-to-SQL. Brown et al. [8] demonstrated GPT-3's capability, a method now vital for complex queries with models like GPT-4 [9] and LLaMA [10]. Studies like Liu et al. [11] further highlight LLMs' zero-shot Text-to-SQL potential, though limitations in logical reasoning persist. However, no benchmark specifically targets zero-shot scenarios robustly. The BIRD dataset [12], with 12,751 question–SQL pairs across

95 databases and 37 domains, is a complex option, but approximately 30% of its queries suffer from logical misalignments (e.g., mismatched timestamps as shown in Figure 1), data inconsistencies, or grammatical errors [12], impairing evaluation. Such ‘dirty’ data—common in real-life benchmarks—hamper LLM reasoning evaluation, as logical clarity is critical in zero-shot settings where fine-tuning is absent [13]. Current Text-to-SQL benchmarks exhibit growing query ambiguity [14], rendering them less suitable for testing LLM logic. For instance, Spider’s [14] structured questions lack the complexity of real-world multi-table joins [14], a gap widened as LLMs master simpler datasets [15]. Conversational benchmarks like SParC [16] and CoSQL [17] focus on multi-turn interactions but neglect broader zero-shot reasoning assessment.

Question 608
State the detailed content of the comment which was created on 7/19/2010 7:25:47 PM.
Answer
SELECT Text FROM comments WHERE CreationDate = '2010-07-19 19:16:14.0'

Figure 1. Example from the BIRD dev dataset (Question 608) illustrating a logical misalignment. The question requests the comment created on 19 July 2010 at 7:25:47 PM, but the provided SQL answer matches the content created on 19 July 2010 at 19:16:14.0, returning incorrect data. This highlights a common issue in BIRD, where approximately 30% of queries fail to align with their SQL outputs, complicating zero-shot Text-to-SQL evaluation.

This study addresses the critical need for a reliable benchmark to evaluate LLM zero-shot reasoning in Text-to-SQL. We refine the BIRD dataset [12] by meticulously selecting 146 complex, logically sound question–SQL pairs across 11 domains. Our refinement process involves correcting SQL errors (e.g., fixing timestamp mismatches), aligning questions with SQL outputs, and enriching table schemas with contextual information (e.g., data ranges, typical values) to minimize ambiguity and reliance on external knowledge. This curated benchmark facilitates a rigorous assessment of LLM zero-shot reasoning on complex, real-world-like queries while eliminating the noise present in the original data. We evaluate six state-of-the-art LLMs (GPT-4o, GPT-4o-Mini, Qwen-2.5-Instruct, Llama 3 70b, DPSK-v3, O1-Preview) using this refined benchmark and specific prompt strategies, achieving average accuracies ranging from 41.65% to 78.08%. This work provides a robust foundation for evaluating and advancing reliable Text-to-SQL systems, supporting practical NLP-driven database applications.

2. Related Works

High-quality datasets are crucial for training and evaluating large language models (LLMs) [8]. Foundational datasets in the Text-to-SQL field include WikiSQL [1], ATIS [18], SParC [16], and CoSQL [17]. WikiSQL focuses on simple queries derived from Wikipedia tables [1], lacking complexity like JOINS [2]. ATIS offers examples in the air travel domain but its simple queries limit generalizability [18]. SParC emphasizes conversational parsing across topics [16], while CoSQL targets dialogue scenarios using Spider’s databases [17,19]. These datasets, while foundational, are increasingly inadequate for modern LLMs due to limited complexity and scope [7].

The advent of neural approaches marked a shift from rule-based systems. Seq2SQL [1] introduced reinforcement learning, followed by SQLNet [2], which improved efficiency without RL. SyntaxSQLNet [3] enhanced cross-domain performance using syntax tree networks. These methods laid the groundwork for LLM-based solutions like DAIL-SQL [4]

and SQLPaLM [5], leveraging pretraining and dynamic context learning [6]. PICARD [20] introduced constrained decoding to refine autoregressive outputs.

Today, Spider [14] and BIRD [12] are prominent benchmarks. Spider targets complex, cross-domain parsing [14], but LLMs like LLaMA [10] and PaLM [21] have diminished this challenge. Zero-shot prompting, pioneered by GPT3 [8], has gained traction, with GPT-4 [9] and LLaMA excelling without training [11], though logical reasoning gaps persist [22]. Recent advancements include multi-agent frameworks like MAC-SQL [23] and SQLFixAgent [24], chain-of-thought prompting [25], and open-source efforts like CodeS [26]. Techniques such as SEA-SQL [27], QDA-SQL [28], OpenSearch-SQL [29], MCS-SQL [30], and Mag-SQL [31] enhance semantic accuracy, few-shot learning [32], and generative approaches. Surveys like [6,7,33] highlight trends toward real-world complexity, yet ambiguous ‘dirty’ data remain a challenge [23,34]. Our work refines BIRD into a clear, complex benchmark for zero-shot LLM evaluation, building on prompt design insights [35] and addressing data quality issues [9,36] to test reasoning robustly.

3. Challenges

While data from older datasets like Spider are currently too easy for LLMs [14], the more complex BIRD benchmark [12] is used here as a data source to ensure sufficient difficulty. Yet BIRD has various issues that harm its utility for assessing LLMs’ capabilities in SQL reasoning [22]. Key problems identified include

Logical and structural errors in SQL: Mismatches, such as omitting requested fields or providing overly complex SQL [12] (Figure A1), and ambiguities leading to misaligned logic [11] (Figure A2) are prevalent.

Inconsistent data representation: Discrepancies between database descriptions and actual data (e.g., ‘Y/N’ vs. ‘0/1’ in Figure A3) [12] or misleading naming conventions [7] can lead to incorrect reasoning.

Ambiguities and poorly defined questions: Vague questions leading to multiple interpretations [13] (Figure A4) or failure to define terms clearly [25] (Figure A5) cause issues.

Redundancy and unreasonable difficulty tags: The excessive repetition of specific test points [6] (Figure A6) or inaccurate difficulty labeling [12] (Figure A7) can skew evaluations.

Incorrect domain-specific knowledge assumptions: Problems requiring domain knowledge not provided or based on wrong assumptions [37] (Figure A8) render data unsuitable for reasoning assessment.

Evaluation challenges: Questions with multiple plausible answers or undefined criteria make validation difficult [34] (Figure A9).

These numerous issues highlight the necessity for significant refinement of the BIRD dataset to ensure valid benchmarking of LLM reasoning capabilities [35]. Rigorous alignment between questions and SQL, clearer question phrasing, and consistent data representation are crucial. Further examples illustrating these error types and their frequency are provided in Appendix A.

4. Methods

4.1. Dataset Creation

After identifying issues, the next step is to address these found issues. Each question is tested on various LLMs several times, and the generated SQL and original target SQL and their execution result are compared, to evaluate whether there are issues and adjust the queries and target SQL. For those data with serious problems that cannot be fixed, they are simply abandoned.

To create an ideal dataset for zero-shot prompt benchmarking, the following rules are applied in the data evaluation process:

Avoid Mismatch Between Questions and SQL Outputs:

As presented before, SQL often omits fields explicitly requested in the question and sometimes diverges from the intent of the question. Such issues can be addressed by adding or removing the appropriate columns in the SQL and by adding or removing the query terms from the question (e.g., Question 586), as shown in Figure 2.

Question 586
Which user added a bounty amount of 50 to the post title mentioning variance?
Original Answer
<pre>SELECT T3.DisplayName, T1.Title FROM posts AS T1 INNER JOIN votes AS T2 ON T1.Id = T2.PostId INNER JOIN users AS T3 ON T3.Id = T2.UserId WHERE T2.BountyAmount = 50 AND T1.Title LIKE '%variance%'</pre>
New Answer
<pre>SELECT DISTINCT users.DisplayName FROM users JOIN votes ON users.Id = votes.UserId JOIN posts ON votes.PostId = posts.Id WHERE votes.BountyAmount = 50 AND posts.Title LIKE '%variance%'</pre>
Reasoning
The question doesn't require T1.Title (highlighted in original answer), so it should not appear in the answer. By removing it, the consistency between the question and answer is secured.

Figure 2. Example of a mismatch between question and SQL output from the BIRD dataset (Question 586). The question asks for the user (originally emphasized) who added a 50-unit bounty to a post mentioning 'variance', but the original SQL includes the unnecessary T1. Title field (originally highlighted). The new answer removes this, using DISTINCT for precision, aligning the SQL with the question's intent as explained in the reasoning.

Avoid Ambiguities in Questions Leading to Misaligned SQL:

Vague phrasing and ambiguous terms like percent or rate are sometimes inconsistently addressed in SQL and can eventually cause misinterpretations. To resolve the ambiguity, the description of data and the terms in question are transformed into more detailed forms (e.g., Question 1028), as shown in Figure 3.

Original Question 1028
In Scotland Premier League, which away team won the most during the 2010 season?
New Question
In Scotland Premier League, which away team won the most during the 2010 to 2011 season?
Reasoning
The phrase "2010 season" is too vague because in some sports, such as Formula One, the 2010 season occurs only in 2010; but in other sports leagues, such as the NBA, the 2010 season can refer to the 2009–2010 season or the 2010–2011 season. To distinguish this ambiguity, the term "2010 season" (highlighted in original question) was changed to "2010–2011 season" (highlighted in new question), which is the case in most sports leagues.

Figure 3. Example of resolving vague phrasing in the BIRD dataset (Question 1028). The original question uses the ambiguous term '2010 season', which could mean a single year or a split season (e.g., 2009–2010 or 2010–2011), depending on the sport. The new question clarifies this as '2010 to 2011 season', reducing misinterpretation risks, as explained in the reasoning.

Avoid Overly Complex SQL for Simple Queries:

Although simpler answers exist, SQL queries can sometimes be unnecessarily complex. The solution is to compare the SQL generated by LLMs and check if they work better. If the SQL generated by LLMs works better in execution speed, etc., replace the original SQL with the generated SQL, as presented before.

Avoid Redundancies or Missing Logic:

SQLs may contain redundant operations or omit key constraints, leading to incomplete or incorrect results (e.g., Question 835). Adding and removing appropriate terms from the SQL can solve such issues, as shown in Figure 4.

Question 835 Among all superheroes in Marvel Comics, identify the percentage of 'good' superheroes.
Original Answer <pre>SELECT CAST(COUNT(CASE WHEN T3.alignment = 'Good' THEN T1.id ELSE NULL END) AS REAL) * 100 / COUNT(T1.id) FROM superhero AS T1 INNER JOIN publisher AS T2 ON T1.publisher_id = T2.id INNER JOIN alignment AS T3 ON T1.alignment_id = T3.id WHERE T2.publisher_name = 'Marvel Comics'</pre>
New Answer <pre>SELECT (SUM(CASE WHEN a.alignment = 'Good' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)) AS percentage_good_superheroes FROM superhero s JOIN publisher p ON s.publisher_id = p.id LEFT JOIN alignment a ON s.alignment_id = a.id WHERE p.publisher_name = 'Marvel Comics'</pre>
Reasoning The “INNER JOIN” in the original answer (highlighted in original answer) causes heroes without alignment data to be excluded. Instead, using “LEFT JOIN” (highlighted in new answer) ensures all heroes are considered, regardless of whether they have alignment data, correcting the incomplete result.

Figure 4. Example of omitted constraints in SQL from the BIRD dataset (Question 835). The question asks for the percentage of 'good' superheroes in Marvel Comics, but the original SQL's INNER JOIN excludes heroes without alignment data. The new answer uses LEFT JOIN to include all heroes, aligning with the question's intent, as explained in the reasoning.

Avoid Inconsistent Use of Data Representations:

SQL queries sometimes omit data inconsistencies or ambiguities, which may misalign with the questions' intent, as in Question 83 (presented before). To solve this issue, the data description document is carefully checked to ensure no ambiguous, wrong, or missing data descriptions appear in the description documents, as shown in Figure 5.

Avoid over-testing the same issue:

Original questions sometimes examine the same point too frequently. This problem can be solved by reducing the proportion of test questions with the same test point, i.e., by selecting only some of them to be added to the new data set.

Adding Information about Data into Table Schema:

This benchmark focuses on the logic of the LLM, so ensuring the integrity of the data provided to the LLM before testing is critical. To this end, the data description file has been modified by adding two columns of data description, Data Range and Typical

Data, to avoid problems caused by unknown data in the dataset to the LLM, as shown in Figure 6. Data ranges are obtained by capturing the maximum and minimum values in the corresponding columns to give the LLM an idea of the range of the data; typical data are obtained by randomly selecting 5 values in a field to simulate the behavior of a quick read of the database and to give the LLM a better understanding of the data.

Original Database Description for Magnet ”The field is coded as follows: · Y = Magnet - The school is a magnet school and/or offers a magnet program. · N = Not Magnet - The school is not a magnet school and/or does not offer a magnet program. commonsense evidence: Note: Preschools and adult education centers do not contain a magnet school indicator.”
New Database Description for Magnet ”The field is coded as follows: · 1 = Magnet - The school is a magnet school and/or offers a magnet program. · 0 = Not Magnet - The school is not a magnet school and/or does not offer a magnet program. commonsense evidence: Note: Preschools and adult education centers do not contain a magnet school indicator.”

Figure 5. Example of correcting data description inconsistency for Question 83 in the BIRD dataset. The original description specifies ‘Y’ and ‘N’ for the ‘Magnet’ field (originally highlighted), but the actual data use ‘0’ and ‘1’. The new description aligns with the data, resolving the ambiguity to ensure SQL queries (e.g., T2. Magnet = 1) match the question’s intent without misinterpretation.

Data Range and Typical Data of Column ‘CDSCode’ from Table ‘frpm’ in Domain ‘California schools’ Data Range: [(‘58727695838305’, ‘01100170109835’)] Typical Data: [(‘36678506111728’,), (‘56724546060321’,), (‘15638181530138’,), (‘31103140119214’,), (‘37683380114520’,)]
--

Figure 6. Example of enhanced data description for the ‘CDSCode’ column in the ‘frpm’ table (California schools domain) from the refined BIRD dataset. The Data Range shows the minimum and maximum values (‘01100170109835’ to ‘58727695838305’), while Typical Data provides five random samples, aiding LLMs in understanding data structure and range. This modification ensures data integrity for logical reasoning evaluation.

Assigning Difficulty Level Properly:

Most test questions were assigned the same difficulty as the original to show fidelity to the original data. Only those test questions that evidently contradicted their own difficulty labels were modified, as shown in Question 322 in the ‘Redundant and Unreasonable Difficulty Labels’ section.

4.2. Testing Method

The testing structure, as illustrated in Figure 7, was specifically designed to assess the reasoning and logic capabilities of the LLMs. During testing, SQL-structure-related exceptions, such as those observed in GPT-4o-Mini (e.g., incorrect marks leading to exceptions), were not regarded as critical for our evaluation. To minimize the impact of such exceptions on the analysis, we applied a secondary verification mechanism in the testing framework. In cases where an exception occurred during the initial attempt, the task will be evaluated again, and only if the exception remains in the second attempt, it will be recorded as a meaningful error and categorized alongside other incorrect cases, as multiple errors on the same task may reveal critical issues.

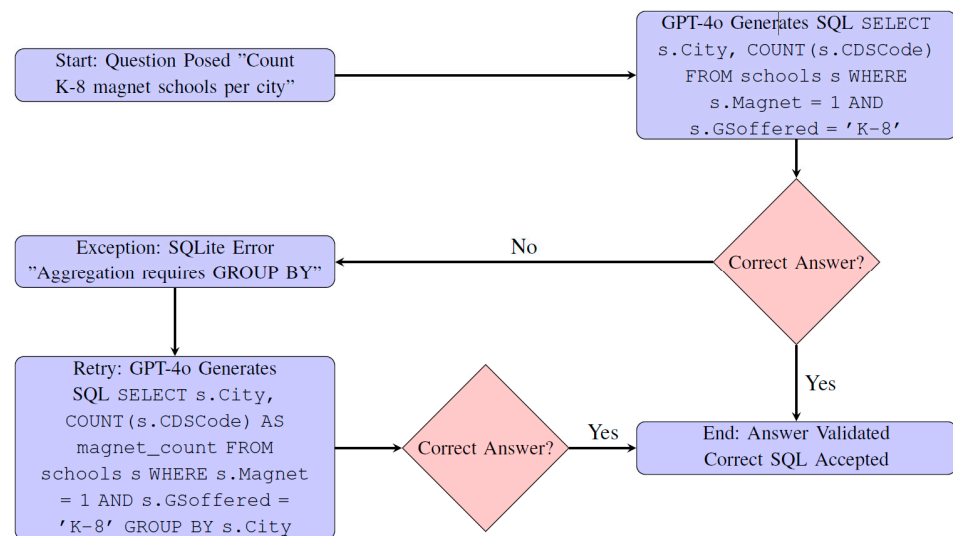


Figure 7. Testing structure with an example: ‘Count the number of K-8 magnet schools in each city’ (California_schools domain). GPT-4o initially generates an incorrect SQL missing GROUP BY, triggers an SQLite exception, retries with the correct SQL, and validates the answer, illustrating the exception handling process in the testing framework.

We conducted two types of experiments in this study. The first experiment involved running each model for five rounds on the entire dataset to calculate their average accuracy. This experiment was conducted on GPT-4o-Mini, GPT-4o, Qwen 2.5 Instruct, LLaMA-3-70B, DPSK-v3, and O1-Preview, with each model evaluated over five rounds on the entire dataset to calculate their average accuracy, aiming to assess their overall reasoning capabilities in zero-shot prompting scenarios in the Text-to-SQL domain.

The second experiment focused on repeated testing of the GPT-4o-Mini, GPT-4o and O1 on tasks that they never answered correctly, respectively. This experiment was designed to analyze the key challenges and limitations faced by the models in T2SQL field.

4.3. Prompt Generation

4.3.1. General Guidelines and Table Schema in Prompt

As shown in Figure 8, to ensure that the test structure functions correctly and allows for automated evaluation, the LLM was required to generate answers in a fixed format. This allows the SQL query within the answer to be easily extracted and executed to determine its correctness. Furthermore, to rigorously test only the reasoning ability, data integrity was prioritized. All necessary data, including relevant table schemas (enhanced as described in Section 4.1), were provided to the LLM via the prompt.

Rules:

1. **Database Type:** The databases are ‘.sqlite’, so use SQLite grammar instead of MySQL grammar.
2. **Column Names with Spaces:** If the column name contains spaces (e.g., “xxx xxx”), use backticks like ‘xxx xxx’ as the column name.
3. **Output Format:** Please answer the user’s question in the following format, including the leading and trailing “” and “”, and “;” at the end of the SQL command:


```

...
Evidence: <reasoning your decision according to the columns
in the table>
ANSWER: <SQL command>;
...
```

Figure 8. Fixed format rules for LLM-generated SQL answers in the refined BIRD benchmark. These rules ensure automated extraction and execution: (1) SQLite grammar matches the ‘.sqlite’ database type, (2) backticks handle spaced column names (e.g., ‘xxx xxx’), and (3) a structured output with evidence and a semicolon-terminated SQL command standardizes responses for correctness validation.

4.3.2. Extra Guidelines in Prompt

Based on common errors observed during initial testing iterations, we introduced additional guidelines into the prompt to mitigate these issues and better isolate core reasoning challenges in zero-shot prompting. These guidelines, detailed in Figures 9–11, address specific problems like handling ties in ranking, data type precision, NULL values, and avoiding unfounded assumptions. They were designed based on common errors and paraphrased for clarity. It is important to note that while these guidelines offer general advice, they do not provide task-specific input–output examples related to the benchmark questions. For instance, advising on NULL or MAX/MIN handling is general SQL knowledge, not specific to any particular question in our dataset. Therefore, we maintain that this setup constitutes a zero-shot scenario, as the model does not learn from task-specific demonstrations but rather operates with enhanced general instructions and context, akin to how a developer might provide documentation or standard practices. Experiments were conducted both with and without these extra guidelines to assess their impact.

Important Guidelines:

- **Accurate Column Selection:**
 - Only select the columns explicitly requested in the user’s query.
 - Do not include extra columns used for sorting or filtering unless they are specifically requested.
 - Avoid over-selection to ensure the result set contains only the necessary information.
- **Use DISTINCT When Necessary:**
 - If the query requires unique records or there’s a possibility of duplicates, use the DISTINCT keyword to eliminate duplicate results.
 - Ensure that DISTINCT is applied appropriately to the correct columns.
- **Handle Logical Operators Correctly:**
 - Pay close attention to logical operators such as AND and OR in the user’s query.
 - Ensure conditions are combined correctly in the SQL statement to match the user’s intent.

Figure 9. The first set of prompt guidelines (Part 1 of 3) for zero-shot Text-to-SQL in the refined BIRD benchmark. These rules tackle column selection ambiguities and bring down the occurrence of common errors, e.g., missing DISTINCT and misuse of OR and AND.

Important Guidelines (continued):

- **Consider Tied Results:**
 - When dealing with MAX, MIN, TOP, or ranking queries, consider that multiple records may share the same value.
 - Include all relevant records that meet the criteria, accounting for possible ties.
- **Data Types and Precision:**
 - Be mindful of data types when performing calculations.
 - Use appropriate type casting or functions to prevent precision loss, especially with integer division.
 - Ensure calculations yield accurate and precise results.
- **Handle Special Data Formats:**
 - If the data contains special formats (e.g., dates, times), use appropriate SQL functions to handle them correctly.
 - Consider edge cases, such as leap years or time zones, if relevant.
- **Avoid Assumptions Based on Common Sense:**
 - Rely on the provided table schemas and data descriptions.
 - Do not make assumptions that contradict the given information, even if it seems counterintuitive.

Figure 10. Second set of prompt guidelines (Part 2 of 3) for zero-shot Text-to-SQL in the refined BIRD benchmark. These rules tackle tied results in ranking queries, data type precision, special format handling (e.g., dates), and avoidance of unfounded assumptions, addressing errors that obscure LLM reasoning capabilities.

Important Guidelines (continued):

- **Combine Conditions Appropriately:**
 - Where possible, combine multiple conditions into a single SQL query rather than splitting them into separate queries.
 - Use subqueries or joins as needed to meet complex query requirements.
- **Null and Empty Data Handling:**
 - Consider the possibility of NULL or missing data.
 - Use appropriate SQL functions (e.g., COALESCE, IS NULL) to handle such cases.
 - Ensure that queries do not fail due to unhandled NULL values.
- **Thorough Reasoning:**
 - In the Evidence section, provide a detailed explanation of your reasoning.
 - Explain how you interpret the user's request and map it to the table columns.
 - Mention how you handle potential issues like duplicates, ties, or special data formats.

Figure 11. Third set of prompt guidelines (Part 3 of 3) for zero-shot Text-to-SQL in the refined BIRD benchmark. These rules focus on combining conditions efficiently, handling NULL values with functions like COALESCE, and providing thorough reasoning, ensuring robust SQL generation and clarity in zero-shot prompting evaluation.

5. Results

5.1. Experimental Setup

All experiments were conducted on a high-performance computing platform using standardized software environments including Python 3.9.7, PyTorch 2.0.1, SQLite 3.36.0, pandas 1.5.3, OpenAI API 1.2.0, and Hugging Face Transformers 4.35.0. The refined dataset

was hosted in SQLite databases. The overall testing framework is illustrated in Figure 7. This figure depicts the process using an example query, showing how initial errors might trigger re-evaluation before final validation. API calls were rate-limited with a timeout. This setup provided a robust environment for assessment.

5.2. Average Accuracy

The performance of six models was evaluated over five rounds on our refined dataset (146 pairs) using new (detailed guidelines) and old (basic guidelines) prompts. Table 1 presents accuracy results compared against a 146-pair subset of the original BIRD dataset. On the refined dataset with the new prompt, accuracies ranged from 41.65% (GPT-4o-Mini) to 78.22% (O1-Preview), outperforming the old prompt. Compared to the original BIRD subset, our dataset yielded gains of 5.35–8.72% (new prompt) and 5.60–8.09% (old prompt). These gains reflect the refinement’s mitigation of BIRD’s issues, while the prompt impact suggests data quality outweighs prompt details, supporting the hypothesis that clean data enhances zero-shot reasoning. This underscores the importance of data quality for reliable zero-shot evaluation, impacting assessments of model generalization and reliability, as discussed further below.

Table 1. Average accuracy of six models across five test rounds on the refined dataset (new and old prompts) compared to the original bird subset. The refined dataset consistently outperforms BIRD, with new prompt accuracies ranging from 41.65% (GPT-4o-mini) to 78.22% (o1-preview), and old prompt accuracies ranging from 36.85% (GPT-4o-mini) to 75.89% (o1-preview).

Models	Refined Dataset (New Prompt)						Refined Dataset (Old Prompt)						Original BIRD	
	Test 1	Test 2	Test 3	Test 4	Test 5	Avg (New)	Test 1	Test 2	Test 3	Test 4	Test 5	Avg (Old)	Avg (New)	Avg (Old)
GPT-4o-Mini	42.47%	46.58%	39.73%	41.78%	37.67%	41.65%	40.41%	32.88%	39.73%	34.93%	36.30%	36.85%	34.12%	31.25%
GPT-4o	47.94%	53.42%	51.36%	53.42%	50.00%	51.23%	50.00%	50.00%	52.05%	42.46%	52.74%	49.45%	43.68%	41.10%
Qwen-2.5-Instruct	46.58%	42.47%	45.21%	41.10%	45.89%	44.25%	43.84%	43.84%	44.52%	46.57%	36.30%	43.01%	38.90%	37.12%
LLaMA-3-70B	47.26%	50.00%	46.58%	45.21%	47.95%	47.80%	46.58%	47.95%	45.21%	44.52%	43.84%	45.62%	41.23%	39.80%
DPSK-v3	48.63%	50.00%	51.37%	47.26%	48.63%	49.10%	47.26%	48.63%	50.00%	46.58%	46.58%	47.80%	42.80%	41.50%
O1-Preview	77.40%	79.45%	76.03%	80.14%	78.08%	78.22%	75.34%	77.40%	74.66%	76.71%	75.34%	75.89%	69.50%	67.80%

5.3. Ablation Studies

To quantify contributions of logical alignment, schema enhancement, and prompt guidelines, we conducted ablation studies across models and prompts on our refined dataset (146 pairs), measuring accuracy and error counts. Starting from full refinement, we incrementally removed components down to the original BIRD subset baseline. Table 2 consolidates accuracy and error distributions. Subsequent analyses explore impacts on reasoning, generalization, and error reduction.

Table 2. Ablation study combining accuracy across models and prompt types with error counts by type for gpt-4o (new prompt). T1–T7 denote error types (T1 Misinterpretation, T2 Forgetting, T3 Syntax, T4 Data Processing, T5 Query Logic, T6 Edge Cases, T7 False Assumptions).

Configuration	Accuracy (%)				Errors by Type (GPT-4o, New Prompt)						
	GPT-4o (New)	GPT-4o (Old)	GPT-4o-Mini (New)	O1-Preview (New)	T1	T2	T3	T4	T5	T6	T7
Full Refinement	51.23	49.45	41.65	78.08	12	5	5	10	24	8	3
w/o Prompt Guidelines	49.80	47.67	40.00	76.85	12	6	6	10	25	9	3
w/o Schema Enhancement	48.12	46.30	38.90	74.52	14	8	5	13	26	8	3
w/o Logical Alignment	45.06	43.15	36.25	70.89	15	10	6	12	32	9	4
Original BIRD Subset	43.68	41.10	34.12	68.49	16	12	7	14	35	10	4
Δ from Full (GPT-4o, New)	-	−1.78	−9.58	+26.85	+4	+7	+2	+4	+11	+2	+1

1. **Impact of Logical Alignment:** Removing logical alignment caused the largest accuracy drop across all models (-6.17% to -7.19%). For GPT-4o (new), Table 2 shows Type 5 errors (query logic) rising significantly, contributing $\sim 82\%$ of the total gain over BIRD. Errors increased substantially in domains like formula_1 and california_schools. This underscores how resolving logical misalignments [12] is pivotal for improving baseline reasoning accuracy, thus enhancing the model's reliability and generalization potential on logically complex tasks.
2. **Impact of Schema Enhancement:** Excluding schema enhancement reduced accuracy (-2.75% to -3.56%). For GPT-4o (new), Type 2 (forgetting requirements) and Type 4 (data handling) errors rose, reflecting reduced guesswork. Domain errors also increased. This supports the role of schema enhancement in reducing guesswork and improving cross-domain generalization by providing clearer data context, leading to more reliable outputs.
3. **Impact of Prompt Guidelines:** Dropping prompt guidelines lowered accuracy moderately (-1.23% to -1.78%). For GPT-4o (new), Type 3 (syntax) and Type 6 (edge cases) errors increased slightly. Domain impacts were smaller. This indicates that while data quality is primary, prompt guidelines play a significant secondary role in fine-tuning reasoning and improving precision, contributing to overall model reliability across different models.

Overall Analysis: The total accuracy gain over BIRD (7.53–9.59%) matches the main experimental trends. (1) For GPT-4o (new), logical alignment drove $\sim 82\%$, schema enhancement $\sim 41\%$, and prompts $\sim 19\%$ of the uplift. (2) Errors increased upon removing refinements confirm BIRD's ambiguities degrade performance. (3) O1-Preview's higher baseline and resilience highlight its superior reasoning, while GPT-4o-Mini's drops underscore sensitivity to data quality. (4) The dominant contribution of logical alignment confirms that clean, unambiguous data are the most critical factor for robust zero-shot performance, directly impacting model generalization and reliability.

5.4. Cross-Round Accuracy

Cross-round accuracy retested persistent errors, showing improved performance on our dataset versus BIRD. O1-Preview's high accuracy and error drop reflect robust reasoning, bolstered by logical alignment. Figures 12–14 provide further granularity on model performance across different domains, difficulty levels, and error types, respectively.

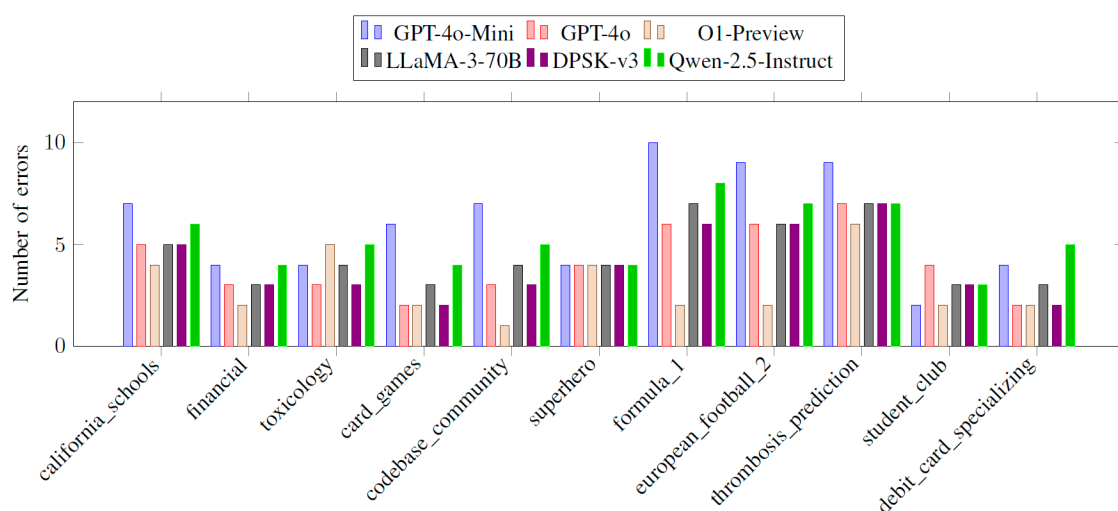


Figure 12. Error comparison across domains for six models.

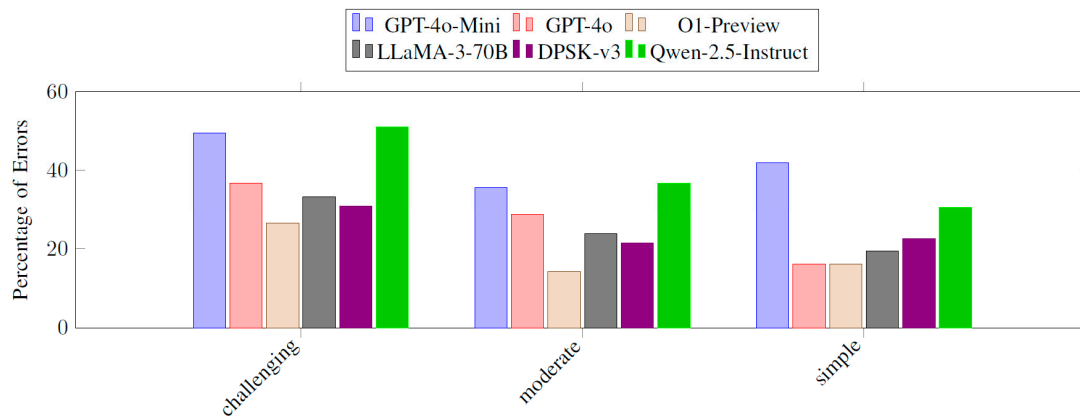


Figure 13. Error percentage by difficulty level for six models.

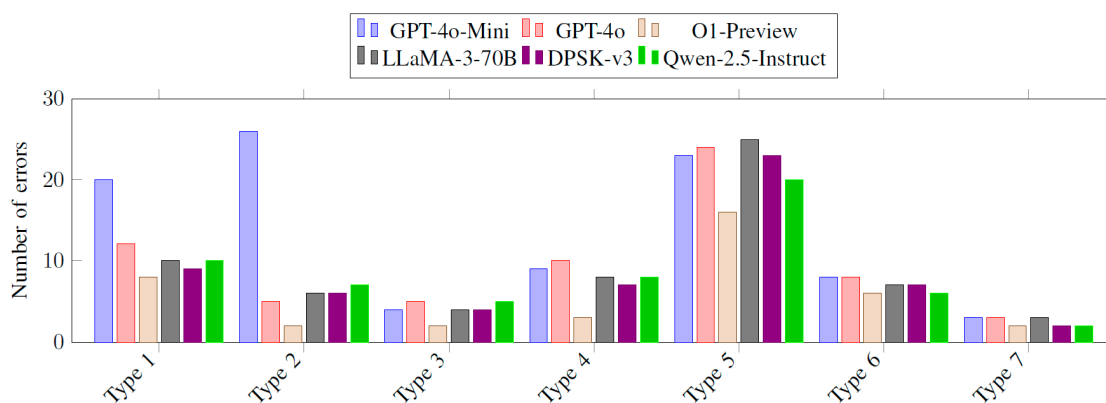


Figure 14. Error comparison by type for six models.

1. Domain Analysis: Figure 12 shows O1-Preview's edge, particularly in complex domains.
2. Difficulty Analysis: Figure 13 highlights O1-Preview's robustness on challenging tasks compared to others.
3. Error Analysis: Figure 14 details seven error types (T1–T7). On BIRD, errors rose significantly. Type 5 (query logic) dominated but was cut by 33% with logical alignment. O1-Preview's performance confirms refinement enhances reasoning. While this error typing provides more insight than a simple pass/fail metric, future work could explore even more granular partial correctness evaluations.

5.5. Case Studies

To illustrate effectiveness, we present three case studies comparing performance on original BIRD versus refined counterparts, highlighting how logical alignment, schema enhancement, and prompt guidelines improve accuracy.

1. Case Study 1: Logical Alignment in formula_1 Domain (Question 880, Figure A4 vs. Figure 15): Refinement clarified ambiguity and aligned SQL with intent, improving accuracy (GPT-4o: 43.68% to 51.23%) and showcasing O1-Preview's optimized solution, validating logical alignment's gain.
2. Case Study 2: Schema Enhancement in the california_schools Domain (Question 83, Figure A3 vs. Figure 16): Refining schema (Figure 5) resolved data inconsistencies, enabling correct interpretation and contributing to accuracy gains by reducing data handling errors.

Refined Question Calculate the percentage difference in Paul di Resta's fastest lap time between races 853 and 854.
GPT-4o Output (Original) <pre>SELECT (SUM(IIF(T2.raceId = 853, T2.<u>fastestLapSpeed</u>, 0)) - SUM(IIF(T2.raceId = 854, T2.<u>fastestLapSpeed</u>, 0))) * 100 / SUM(IIF(T2.raceId = 853, T2.<u>fastestLapSpeed</u>, 0)) FROM drivers AS T1 INNER JOIN results AS T2 ON T1.driverId = T2.driverId WHERE T1.forename = 'Paul' AND T1.surname = 'di Resta';</pre> <i>(Erroneous: Sums speed instead of time)</i>
GPT-4o Output (Refined) <pre>SELECT ((SELECT MAX(fastestLapTime) FROM results WHERE raceId = 853 AND driverId = (SELECT driverId FROM drivers WHERE forename = 'Paul' AND surname = 'di Resta')) - (SELECT MAX(fastestLapTime) FROM results WHERE raceId = 854 AND driverId = (SELECT driverId FROM drivers WHERE forename = 'Paul' AND surname = 'di Resta')) * 100.0 / (SELECT MAX(fastestLapTime) FROM results WHERE raceId = 853 AND driverId = (SELECT driverId FROM drivers WHERE forename = 'Paul' AND surname = 'di Resta'));</pre> <i>(Correct: Uses time with MAX)</i>
O1-Preview Output (Refined) <pre>SELECT ((r1.fastestLapTime - r2.fastestLapTime) * 100.0 / r1.fastestLapTime) AS percentage_difference FROM results r1, results r2, drivers d WHERE r1.raceId = 853 AND r2.raceId = 854 AND r1.driverId = d.driverId AND r2.driverId = d.driverId AND d.forename = 'Paul' AND d.surname = 'di Resta' AND r1.fastestLapTime = (SELECT MAX(fastestLapTime) FROM results WHERE raceId = 853 AND driverId = d.driverId) AND r2.fastestLapTime = (SELECT MAX(fastestLapTime) FROM results WHERE raceId = 854 AND driverId = d.driverId);</pre> <i>(Optimized: Join-based with time)</i>

Figure 15. Case Study 1: Logical alignment resolves ambiguity in formula_1. The original GPT-4o output **sums speeds**, missing the intent, while refined outputs correctly compute the **percentage difference** of **fastest lap times**. O1-Preview's concise join-based solution aligns with its 80% error reduction in this domain (Cross-round Accuracy).

Refined Question Count K-8 magnet schools (Magnet = 1) per city with multiple NSLP provision types.
GPT-4o Output (Original) <pre>SELECT T2.City, COUNT(T2.CDSCode) FROM frpm AS T1 INNER JOIN schools AS T2 ON T1.CDSCode = T2.CDSCode WHERE T2.Magnet = 1 AND T2.GSoffered = 'K-8' AND T1.'NSLP Provision Status' = 'Multiple Provision Types' GROUP BY T2.City;</pre> <i>(Risky: Assumes 0/1 coding despite 'Y/N' description)</i>
GPT-4o Output (Refined) <pre>SELECT s.City, COUNT(s.CDSCode) AS magnet_count FROM schools s JOIN frpm f ON s.CDSCode = f.CDSCode WHERE s.Magnet = 1 AND s.GSoffered = 'K-8' AND f.'NSLP Provision Status' = 'Multiple Provision Types' GROUP BY s.City;</pre> <i>(Correct: Matches 0/1 schema)</i>
O1-Preview Output (Refined) <pre>SELECT s.City, COUNT(DISTINCT s.CDSCode) AS magnet_count FROM schools s JOIN frpm f ON s.CDSCode = f.CDSCode WHERE s.Magnet = 1 AND s.GSoffered = 'K-8' AND f.'NSLP Provision Status' = 'Multiple Provision Types' GROUP BY s.City;</pre> <i>(Enhanced: Adds DISTINCT for precision)</i>

Figure 16. Case Study 2: Schema enhancement in california_schools. Original GPT-4o output risks errors with **Magnet** under 'Y/N' description. Refined schema ensures 0/1 consistency, and O1-Preview's **DISTINCT** cuts errors from 5 to 4 (Figure 12).

- Case Study 3: Prompt Guidelines in the superhero Domain (Question 835, Figure 4 vs. Figure 17): Guidelines prompted the correct use of LEFT JOIN and NULL handling, correcting omissions and improving accuracy by reducing edge case errors.
- These case studies confirm that our method resolves BIRD's issues, enhancing LLM reasoning and supporting accuracy uplift.
- Improvements in Runtime Performance: Refinements also enhance SQL execution efficiency (1). For instance, simplifying Question 477 (Figure A1) reduced execution time by 65.16% (2). We measured the times for 20 representative pairs across models (3). Figure 18 shows that refined SQL consistently reduces execution time (e.g., GPT-4o's average drop of 46.12%) (4), particularly in complex domains (5). Improvements stem from logical alignment and prompt guidelines reducing computational overhead (6). Schema enhancements have a minimal direct runtime impact (7). This demonstrates practical runtime gains critical for real-world applications (8).

Refined Question What percentage of Marvel Comics superheroes are good, including those with missing alignment ?
GPT-4o Output (Original) <pre>SELECT CAST(COUNT(CASE WHEN T3.alignment = 'Good' THEN T1.id ELSE NULL END) AS REAL) * 100 / COUNT(T1.id) FROM superhero AS T1 INNER JOIN publisher AS T2 ON T1.publisher_id = T2.id INNER JOIN alignment AS T3 ON T1.alignment_id = T3.id WHERE T2.publisher_name = 'Marvel Comics';</pre> <i>(Erroneous: INNER JOIN excludes unaligned heroes)</i>
GPT-4o Output (Refined) <pre>SELECT (SUM(CASE WHEN a.alignment = 'Good' THEN 1 ELSE 0 END) * 100.0 / COUNT(*)) AS percentage_good FROM superhero s JOIN publisher p ON s.publisher_id = p.id LEFT JOIN alignment a ON s.alignment_id = a.id WHERE p.publisher_name = 'Marvel Comics';</pre> <i>(Correct: LEFT JOIN includes all heroes)</i>
O1-Preview Output (Refined) <pre>SELECT (SUM(CASE WHEN a.alignment = 'Good' THEN 1 ELSE 0 END) * 100.0 / COUNT(s.id)) AS percentage_good FROM superhero s LEFT JOIN alignment a ON s.alignment_id = a.id JOIN publisher p ON s.publisher_id = p.id WHERE p.publisher_name = 'Marvel Comics';</pre> <i>(Consistent: LEFT JOIN with NULL handling)</i>

Figure 17. Case Study 3: Prompt guidelines in superhero. Original GPT4o uses **INNER JOIN**, excluding unaligned heroes, while refined outputs apply **LEFT JOIN** and NULL handling per guidelines, stabilizing errors at 4 (Figure 12), boosting accuracy by 1.43% (Ablation Studies).

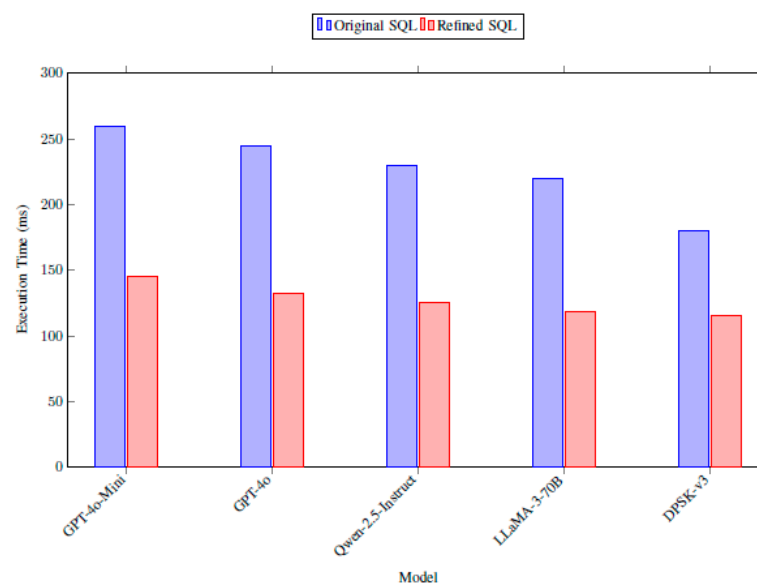


Figure 18. Comparison of average SQL execution times (ms) for original vs. refined queries across five models, based on 20 representative tasks from the refined dataset. Refined SQL consistently outperforms, with reductions ranging from 36.11% (DPSK-v3) to 46.12% (GPT-4o).

6. Conclusions

This study established a robust dataset by addressing flaws in BIRD [12], enabling a more accurate assessment of LLMs' logical capabilities in zero-shot prompting. Performance aligned with model trajectories, validating the dataset. We identified common LLM issues and showed that prompt-based techniques offer mitigation, improving success rates and providing insights. Error-annotated datasets facilitate further analysis. These contributions enhance understanding and establish a foundation for ongoing improvements.

Despite progress, limitations remain: (1) Benchmarking is currently limited to SQLite; performance across other SQL dialects needs investigation. (2) Evaluations should encompass a broader range of LLMs. (3) Furthermore, future work could incorporate more granular evaluation metrics beyond execution accuracy, potentially exploring partial correctness scores, and employ rigorous statistical tests (e.g., ANOVA, t-tests) to validate performance differences. Finally, while our refined benchmark aims to reduce artifacts, the risk of overfitting exists; validation in diverse, real-world downstream applications, particularly critical decision-making systems, is essential to ensure true robustness and reliability.

Author Contributions: Conceptualization, F.Z.; methodology, R.Z.; software, R.Z.; validation, R.Z., and F.Z.; investigation, F.Z.; writing—original draft preparation, R.Z.; writing—review and editing, F.Z.; supervision, F.Z.; funding acquisition, F.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This work is supported by the Ministry of Science and Technology of the People's Republic of China (Grant No. 2023ZD0120704 under Project No. 2023ZD0120700) and the National Natural Science Foundation of China (Grant No. 62372409).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflict of interest.

Appendix A. Examples of Errors and Inconsistencies in the Original BIRD Dataset

This appendix provides further details and context for the types of errors and inconsistencies identified in the original BIRD dataset, as discussed in Section 3. These examples illustrate the challenges that necessitate dataset refinement for reliable zero-shot Text-to-SQL benchmarking.

Appendix A.1. Logical and Structural Errors in SQL

Example (Overly Complex SQL—Figure A1): Question 477 asks for artists who designed a specific card. The original SQL provided in BIRD uses repetitive OR conditions combined with GROUP BY, resulting in inefficient execution (287 ms). A simpler, logically equivalent query using IN and DISTINCT achieves the same result much faster (100 ms). This indicates the original dataset sometimes provides suboptimal or unnecessarily complex ground truth SQL.

Example (Misaligned SQL Logic—Figure A2): Question 94 asks for account numbers of female clients who meet specific criteria (oldest with lowest average salary) and the salary gap. However, the provided SQL incorrectly filters based on the district of a single oldest female client, failing to capture the broader intent of identifying relevant clients across districts based on combined age and salary criteria. This misalignment stems from ambiguity in the question, potentially leading to incorrect SQL logic.

Appendix A.2. Inconsistent Data Representation

Example (Data Value Mismatch—Figure A3): Question 83 involves filtering based on a ‘Magnet’ school indicator. The database description file specifies this field uses ‘Y’/‘N’ values. However, the actual data in the table use ‘0’/‘1’, and the provided SQL query correctly uses `T2. Magnet = 1`. This inconsistency between the schema description and the actual data can mislead LLMs that rely on the description.

Example (Misleading Naming): In some cases, table or column names mentioned in the database description documents were found to be inconsistent with the actual schema (e.g., ‘set transaction’ instead of ‘set translation’), potentially causing LLMs to fail in identifying the correct tables or columns.

Appendix A.3. Ambiguities and Poorly Defined Questions

Example (Vague Phrasing—Figure A4): Question 880 asks for a percentage difference related to ‘the fastest lap speed’. The term ‘fastest lap speed’ is singular, but the provided SQL calculates a percentage difference based on the *sum* of fastest lap speeds across races, misinterpreting the likely intent. Such vagueness can lead to multiple interpretations and incorrect SQL.

Example (Ambiguous Target—Figure A5): Question 198 asks ‘On average how many carcinogenic molecules are single bonded?’. The phrasing suggests counting molecules, but the provided SQL calculates the average *number of single bonds* per carcinogenic molecule. The intended question was likely different, highlighting how unclear definitions lead to misaligned SQL.

Appendix A.4. Redundancy and Unreasonable Difficulty Tags

Example (Redundant Testing—Figure A6): Questions 299, 302, 319, and several others (at least 12 identified) repeatedly test the LLM’s understanding of using `DISTINCT` when counting ‘elements’. While valid, such excessive repetition of a single concept leads to unbalanced evaluation.

Example (Difficulty Mislabeling—Figure A7): Question 322, which requires a simple lookup of an element by its ID, is labeled as ‘challenging’ in the dataset. This clear overestimation of difficulty is misleading and can skew evaluation metrics.

Appendix A.5. Incorrect Domain-Specific Knowledge Assumptions

Example (Flawed Domain Assumption—Figure A8): Question 416 asks for the percentage of cards ‘without power’ in a specific language (French) within the context of Magic: The Gathering. The provided SQL incorrectly includes cards with power ‘*under the ‘without power’ condition. In MTG,*’ denotes variable power, not zero power. This reflects an incorrect domain-specific assumption embedded in the ground truth SQL, making it unsuitable for evaluating reasoning.

Appendix A.6. Evaluation Challenges

Example (Undefined Evaluation Criteria—Figure A9): Question 1058 asks ‘Who has the highest average finishing rate between the highest and shortest football player?’. The SQL returns ‘Max’ or ‘Min’, but the definition of ‘who’ is unclear—it could plausibly be the player’s name or a comparative term like ‘higher’ or ‘lower’. This lack of explicit output format definition makes automated validation difficult or ambiguous.

Appendix A.7. Summary of Identified Error Types in Original BIRD Dataset

The following table summarizes the main categories of issues identified within the original BIRD dataset during our analysis, referencing examples discussed above. While

the paper estimates roughly 30% of pairs suffer from issues, this table categorizes the *types* of problems encountered.

Error Category	Description	Example Figure(s)
Logical/Structural SQL Errors	The SQL query does not accurately reflect the question's logic, or the SQL structure is inefficient/suboptimal.	Figures A1 and A2
Data Representation Issues	Inconsistencies between schema descriptions and actual data values, or misleading naming conventions.	Figure A3
Question Ambiguity	Vague phrasing, undefined terms, or unclear intent in the natural language question leads to ambiguity.	Figures A4 and A5
Dataset Bias Issues	Excessive repetition of specific query types/concepts or inaccurate difficulty labeling.	Figures A6 and A7
Domain Knowledge Issues	Reliance on incorrect or unstated domain-specific assumptions embedded within the question or SQL.	Figure A8
Evaluation Ambiguity	Lack of clear criteria for evaluating the correctness of the output, allowing multiple plausible answers.	Figure A9

Question 477 Which of these artists have designed a card in the set Coldsnap, Jeremy Jarvis, Aaron Miller or Chippy?
Original Answer (Execution Time: 287 ms) <pre>SELECT T1.artist FROM cards AS T1 INNER JOIN sets AS T2 ON T2.code = T1.setCode WHERE (T2.name = 'Coldsnap' AND T1.artist = 'Chippy') OR (T2.name = 'Coldsnap' AND T1.artist = 'Aaron Miller') OR (T2.name = 'Coldsnap' AND T1.artist = 'Jeremy Jarvis') GROUP BY T1.artist</pre>
Newly Found Answer (Execution Time: 100 ms) <pre>SELECT DISTINCT cards.artist FROM cards JOIN sets ON cards.setCode = sets.code WHERE cards.artist IN ('Jeremy Jarvis', 'Aaron Miller', 'Chippy') AND sets.name = 'Coldsnap'</pre>

Figure A1. Example of an overly complex SQL query from the BIRD dataset (Question 477). The question asks which of the listed artists (Jeremy Jarvis, Aaron Miller, Chippy) designed a card in the Coldsnap set. The original answer uses repetitive conditions and a GROUP BY, taking 287 ms, while the simpler, more efficient alternative uses IN and DISTINCT, reducing execution time to 100 ms. This demonstrates a common mismatch where unnecessarily complex SQL is provided.

Question 94 List out the account numbers of female clients who are oldest and has lowest average salary, calculate the gap between this lowest average salary with the highest average salary?
Original Answer <pre>SELECT T1.account_id , (SELECT MAX(A11) - MIN(A11) FROM district) FROM account AS T1 INNER JOIN district AS T2 ON T1.district_id = T2.district_id WHERE T2.district_id = (SELECT district_id FROM client WHERE gender = 'F' ORDER BY birth_date ASC LIMIT 1) ORDER BY T2.A11 DESC LIMIT 1</pre>
Error Analysis The condition in the original answer (highlighted) restricts the query to the district of the single oldest female client. Since the average salary (A11) is uniform within a district, the question's intent—"female clients who are oldest and has lowest average salary"—implies identifying districts with the greatest average age of women and lowest salary, not just one client's district. This misalignment reflects ambiguous phrasing in the question leading to incorrect SQL logic.

Figure A2. Example of ambiguous question phrasing causing misaligned SQL logic from the BIRD dataset (Question 94). The question seeks account numbers of female clients who are the oldest with the lowest average salary, plus the salary gap. The original SQL incorrectly focuses on the district of the single oldest female client, missing the broader intent of aggregating across districts, as explained in the error analysis.

Original Answer for Question 83 <pre>SELECT T2.City, COUNT(T2.CDSCode) FROM frpm AS T1 INNER JOIN schools AS T2 ON T1.CDSCode = T2.CDSCode WHERE T2.Magnet = 1 AND T2.GSoffered = 'K-8' AND T1.'NSLP Provision Status' = 'Multiple Provision Types' GROUP BY T2.City</pre>
Database Description for Magnet "The field is coded as follows: · Y = Magnet - The school is a magnet school and/or offers a magnet program. · N = Not Magnet - The school is not a magnet school and/or does not offer a magnet program. commonsense evidence: Note: Preschools and adult education centers do not contain a magnet school indicator."
Error Analysis The condition <code>T2.Magnet = 1</code> (highlighted in original answer) conflicts with the database description (highlighted in database description). The actual data in the 'Magnet' column uses '0' and '1', indicating the description's 'Y/N' coding is incorrect, leading to potential misreasoning by LLMs.

Figure A3. Example of inconsistency between database description and actual data from the BIRD dataset (Question 83). The original SQL assumes 'Magnet' is coded as '0' and '1' (using `T2.Magnet = 1`), while the database description specifies 'Y' and 'N', causing a mismatch. This error, noted in the analysis, can mislead LLMs into incorrect reasoning, reflecting a broader issue of data inconsistency in BIRD.

Question 880
Paul di Resta was in the No. 853 race, what percent faster did he finish in the 853rd race than the next race for the fastest lap speed?
Original Answer <pre>SELECT (SUM(IIF(T2.raceId = 853, T2.fastestLapSpeed, 0)) - SUM(IIF(T2.raceId = 854, T2.fastestLapSpeed, 0))) * 100 / SUM(IIF(T2.raceId = 853, T2.fastestLapSpeed, 0)) FROM drivers AS T1 INNER JOIN results AS T2 ON T2.driverId = T1.driverId WHERE T1.forename = 'Paul' AND T1.surname = 'di Resta'</pre>
Error Analysis The SQL calculates the percentage increase of the sum of fastest lap speeds (highlighted in question), which may not align with the question's intent. "The fastest lap speed" typically refers to the single fastest lap per race, suggesting the query should compare individual lap speeds, not their totals, leading to potential inconsistency.

Figure A4. Example of a vague question from the BIRD dataset (Question 880). The question asks for the percent faster Paul di Resta's fastest lap speed was in race 853 compared to race 854, but the original SQL sums all lap speeds, misinterpreting the singular 'fastest lap' term. This ambiguity can result in inconsistent or incorrect answers.

Question 198
On average how many carcinogenic molecules are single bonded?
Original Answer <pre>SELECT AVG(single_bond_count) FROM (SELECT T3.molecule_id, COUNT(T1.bond_type) AS single_bond_count FROM bond AS T1 INNER JOIN atom AS T2 ON T1.molecule_id = T2.molecule_id INNER JOIN molecule AS T3 ON T3.molecule_id = T2.molecule_id WHERE T1.bond_type = '-' AND T3.label = '+' GROUP BY T3.molecule_id) AS subquery</pre>
Error Analysis The phrase "on average" creates ambiguity, and the question's target—"how many carcinogenic molecules"—suggests counting molecules, yet the SQL computes the average number of single bonds per carcinogenic molecule (highlighted in original answer). The intended question likely should be "On average, how many single bonds are there in carcinogenic molecules?" revealing a mismatch due to unclear terms.

Figure A5. Example of an ambiguously defined question from the BIRD dataset (Question 198). The question's phrasing suggests counting single-bonded carcinogenic molecules, but the SQL calculates the average number of single bonds per molecule. This discrepancy, noted in the analysis, stems from vague terms, leading to misaligned SQL output.

Question 299
Which toxic element can be found in the molecule TR151?
Question 302
Name the elements that comprise the atoms of bond TR001_2_4.
Question 319
How many elements are contained in bond_id TR001_3_4?
Remark
The purpose of all of these questions is to test whether the LLM recognizes the need for a “DISTINCT” when referring to “elements” since each element can only be counted once. Questions 268, 277, 281, 285, 290, 302, 304, 306, 307, 319, 322, and 328 all address this process, which causes redundancies.

Figure A6. Examples of redundant questions from the BIRD dataset (Questions 299, 302, 319). These questions repeatedly test the LLM’s understanding of ‘element’ and the need for DISTINCT to avoid duplicate counting. The remark notes that at least 12 questions focus on this single concept, leading to unbalanced data that overemphasize one reasoning aspect, reducing evaluation persuasiveness.

Question 322 (Labeled as 299)
What is the element of toxicology for the atom with the ID of TR000_1?
Original Answer
<code>SELECT T.element FROM atom AS T WHERE T.atom_id = 'TR000_1'</code>
Remark
The question is labeled “challenging” but not difficult in any way.

Figure A7. Example of difficulty mislabeling from the BIRD dataset (Question 322, mislabeled as 299). The question, a simple lookup of an element by atom ID, is labeled ‘challenging’ despite its straightforward SQL solution. This overestimation of difficulty can skew evaluation metrics and reduce the dataset’s reliability.

Question 416
What percentage of cards without power are in French?
Original Answer
<code>SELECT CAST(SUM(CASE WHEN T2.language = 'French' THEN 1 ELSE 0 END) AS REAL) * 100 / COUNT(T1.id) FROM cards AS T1 INNER JOIN foreign_data AS T2 ON T1.uuid = T2.uuid WHERE T1.power IS NULL OR T1.power LIKE '%*%'</code>
Error Analysis
In Magic: The Gathering (MTG), a power value of '*' is not considered “without power” (highlighted in question); it indicates variable power. The SQL’s condition (highlighted in original answer) incorrectly includes such cards, showing the dataset developers made a false domain-specific assumption.

Figure A8. Example of an incorrect domain knowledge assumption in the BIRD dataset (Question 416). The question asks for the percentage of cards ‘without power’ in French, but the SQL includes cards with '*' power, which in MTG denotes variable power, not an absence of power. This error, noted in the analysis, renders the data unsuitable for assessing LLM reasoning due to flawed assumptions.

Question 1058
Who has the highest average finishing rate between the highest and shortest football player?
Original Answer
<pre>SELECT A FROM (SELECT AVG(finishing) result, 'Max' A FROM Player AS T1 INNER JOIN Player_Attributes AS T2 ON T1.player_api_id = T2.player_api_id WHERE T1.height = (SELECT MAX(height) FROM Player) UNION SELECT AVG(finishing) result, 'Min' A FROM Player AS T1 INNER JOIN Player_Attributes AS T2 ON T1.player_api_id = T2.player_api_id WHERE T1.height = (SELECT MIN(height) FROM Player)) ORDER BY result DESC LIMIT 1</pre>
Error Analysis
The question lacks explicit guidance on the desired response type (e.g., “Min” or “Max”, highlighted in original answer), allowing multiple valid answers such as the player’s name, “higher”, or “lower” (originally highlighted possibilities). For clarity and evaluability, it should specify that the output is either “Min” or “Max”.

Figure A9. Example of an ambiguous question with undefined evaluation criteria from the BIRD dataset (Question 1058). The question asks who has the highest average finishing rate between the tallest and shortest football players, but the SQL returns ‘Max’ or ‘Min’ without clear standards for ‘who’, permitting alternatives like player names or comparative terms. This ambiguity, noted in the analysis, makes validation infeasible.

References

1. Zhong, V.; Xiong, C.; Socher, R. Seq2SQL: Generating structured queries from natural language using reinforcement learning. *arXiv* **2017**, arXiv:1709.00103.
2. Xu, X.; Liu, C.; Yu, D. SQLNet: Generating structured queries from natural language without reinforcement learning. *arXiv* **2017**, arXiv:1711.04436.
3. Yu, T.; Yasunaga, M.; Yang, K.; Zhang, R.; Wang, D.; Li, Z.; Radev, D. SyntaxSQLNet: Syntax tree networks for complex and cross-domain text-to-SQL task. *arXiv* **2018**, arXiv:1810.05237.
4. Gao, D.; Wang, H.; Li, Y.; Sun, X.; Qian, Y.; Ding, B.; Zhou, J. Text-to-SQL empowered by large language models: A benchmark evaluation. *arXiv* **2023**, arXiv:2308.15363. [[CrossRef](#)]
5. Sun, R.; Arik, S.Ö.; Muzio, A.; Miculicich, L.; Gundabathula, S.; Yin, P.; Dai, H.; Nakhost, H.; Sinha, R.; Wang, Z.; et al. SQL-PaLM: Improved large language model adaptation for text-to-SQL. *arXiv* **2023**, arXiv:2306.00739.
6. Hong, Z.; Yuan, Z.; Zhang, Q.; Chen, H.; Dong, J.; Huang, F.; Huang, X. Next-generation database interfaces: A survey of llm-based text-to-SQL. *arXiv* **2024**, arXiv:2406.08426.
7. Shi, L.; Tang, Z.; Zhang, N.; Zhang, X.; Yang, Z. A survey on employing large language models for text-to-SQL tasks. *arXiv* **2023**, arXiv:2407.15109.
8. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1877–1901.
9. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. Gpt-4 technical report. *arXiv* **2023**, arXiv:2303.08774.
10. Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. Llama: Open and efficient foundation language models. *arXiv* **2023**, arXiv:2302.13971.
11. Liu, A.; Hu, X.; Wen, L.; Yu, P.S. A comprehensive evaluation of chatgpt’s zero-shot text-to-SQL capability. *arXiv* **2023**, arXiv:2303.13547.
12. Li, J.; Hui, B.; Qu, G.; Yang, J.; Li, B.; Li, B.; Wang, B.; Qin, B.; Geng, R.; Huo, N.; et al. Can llm already serve as a database interface? a big bench for large-scale database grounded textto-SQLs. *arXiv* **2023**, arXiv:2305.03111.

13. Dong, X.; Zhang, C.; Ge, Y.; Mao, Y.; Gao, Y.; Lin, J.; Lou, D. C3: Zero-shot text-to-SQL with chatgpt. *arXiv* **2023**, arXiv:2307.07306.
14. Yu, T.; Zhang, R.; Yang, K.; Yasunaga, M.; Wang, D.; Li, Z.; Ma, J.; Li, I.; Yao, Q.; Roman, S.; et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. *arXiv* **2018**, arXiv:1809.08887.
15. Lei, F.; Chen, J.; Ye, Y.; Cao, R.; Shin, D.; Su, H.; Suo, Z.; Gao, H.; Hu, W.; Yin, P.; et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *arXiv* **2024**, arXiv:2411.07763.
16. Yu, T.; Zhang, R.; Yasunaga, M.; Tan, Y.C.; Lin, X.V.; Li, S.; Er, H.; Li, I.; Pang, B.; Chen, T.; et al. Sparc: Crossdomain semantic parsing in context. *arXiv* **2019**, arXiv:1906.02285.
17. Yu, T.; Zhang, R.; Er, H.Y.; Li, S.; Xue, E.; Pang, B.; Lin, X.V.; Tan, Y.C.; Shi, T.; Li, Z.; et al. CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases. *arXiv* **2019**, arXiv:1909.05378.
18. Dahl, D.A.; Bates, M.; Brown, M.; Fisher, W.; Hunicke-Smith, K.; Pallett, D.; Pao, C.; Rudnicky, A.; Shriberg, E. Expanding the scope of the ATIS task: The ATIS-3 corpus. In Proceedings of the Human Language Technology: Proceedings of a Workshop, Plainsboro, NJ, USA, 8–11 March 1994.
19. Kelkar, A.; Relan, R.; Bhardwaj, V.; Vaichal, S.; Khatri, C.; Relan, P. Bertrand-dr: Improving text-to-SQL using a discriminative re-ranker. *arXiv* **2020**, arXiv:2002.00557.
20. Scholak, T.; Schucher, N.; Bahdanau, D. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. *arXiv* **2021**, arXiv:2109.05093.
21. Anil, R.; Dai, A.M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. PaLM 2 technical report. *arXiv* **2023**, arXiv:2305.10403.
22. Rajkumar, N.; Li, R.; Bahdanau, D. Evaluating the text-to-SQL capabilities of large language models. *arXiv* **2022**, arXiv:2204.00498.
23. Wang, B.; Ren, C.; Yang, J.; Liang, X.; Bai, J.; Chai, L.; Yan, Z.; Zhang, Q.W.; Yin, D.; Sun, X.; et al. Mac-SQL: A multi-agent collaborative framework for text-to-SQL. *arXiv* **2024**, arXiv:2312.11242.
24. Cen, J.; Liu, J.; Li, Z.; Wang, J. SQLfixagent: Towards semantic-accurate text-to-SQL parsing via consistency-enhanced multi-agent collaboration. *arXiv* **2024**, arXiv:2406.13408. [[CrossRef](#)]
25. Tai, C.Y.; Chen, Z.; Zhang, T.; Deng, X.; Sun, H. Exploring chain of thought style prompting for text-to-SQL. *Empirical Methods in Natural Language Processing (EMNLP)*. *arXiv* **2023**, arXiv:2305.14215.
26. Li, H.; Zhang, J.; Liu, H.; Fan, J.; Zhang, X.; Zhu, J.; Wei, R.; Pan, H.; Li, C.; Chen, H. Codes: Towards building opensource language models for text-to-SQL. *Conf. Manag. Data (SIGMOD)* **2024**, 2, 1–28.
27. Li, C.; Shao, Y.; Li, Y.; Liu, Z. Sea-SQL: Semantic-enhanced text-to-SQL with adaptive refinement. *arXiv* **2024**, arXiv:2408.04919.
28. Sun, Y.; Guo, Z.; Yu, H.; Liu, C.; Li, X.; Wang, B.; Yu, X.; Zhao, T. Qda-SQL: Questions enhanced dialogue augmentation for multi-turn text-to-SQL. *arXiv* **2024**, arXiv:2406.10593.
29. Xie, X.; Xu, G.; Zhao, L.; Guo, R. Opensearch-SQL: Enhancing text-to-SQL with dynamic few-shot and consistency alignment. *arXiv* **2025**, arXiv:2502.14913.
30. Lee, D.; Park, C.; Kim, J.; Park, H. Mcs-SQL: Leveraging multiple prompts and multiple-choice selection for text-to-SQL generation. *arXiv* **2024**, arXiv:2405.07467.
31. Xie, W.; Wu, G.; Zhou, B. Mag-SQL: Multi-agent generative approach with soft schema linking and iterative sub-SQL refinement for text-to-SQL. *arXiv* **2024**, arXiv:2408.07930.
32. Gu, Z.; Fan, J.; Tang, N.; Cao, L.; Jia, B.; Madden, S.; Du, X. Few-shot text-to-SQL translation using structure and content prompt learning. *Proc. ACM Manag. Data* **2023**, 1, 1–28. [[CrossRef](#)]
33. Zhu, X.; Li, Q.; Cui, L.; Liu, Y. Large language model enhanced text-to-SQL generation: A survey. *arXiv* **2024**, arXiv:2410.05678.
34. Xia, H.; Jiang, F.; Deng, N.; Wang, C.; Zhao, G.; Mihalcea, R.; Zhang, Y. SQL-craft: Text-to-SQL through interactive refinement and enhanced reasoning. *arXiv* **2024**, arXiv:2402.14851.
35. Nan, L.; Zhao, Y.; Zou, W.; Ri, N.; Tae, J.; Zhang, E.; Cohan, A.; Radev, D. Enhancing text-toSQL capabilities of large language models: A study on prompt design strategies. In *Findings of Empirical Methods in Natural Language Processing (EMNLP)*; Association for Computational Linguistics: Singapore, 2023.
36. Wu, Z.; Zhu, F.; Shang, X.; Zhang, Y.; Zhou, P. Cooperative SQL generation for segmented databases by using multi-functional llm agents. *arXiv* **2024**, arXiv:2412.05850.
37. Chang, S.; Fosler-Lussier, E. Selective demonstrations for crossdomain text-to-SQL. *arXiv* **2023**, arXiv:2310.06302.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.